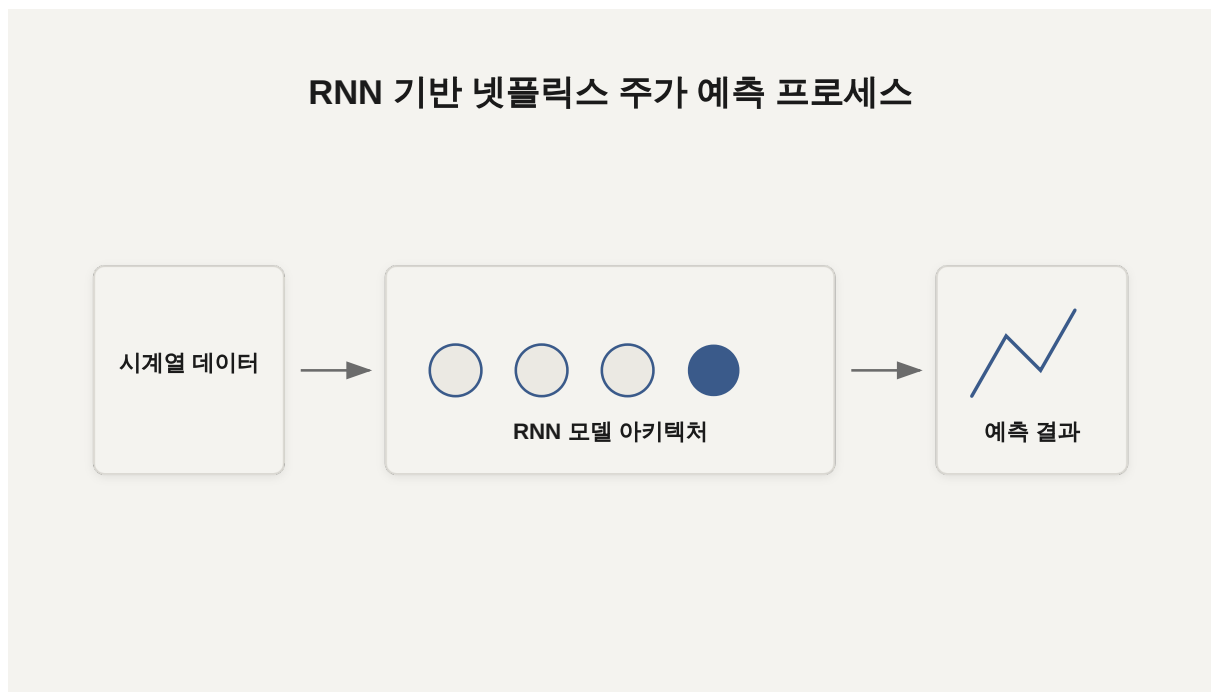


STEP 3. RNN/LSTM (1주)

넷플릭스 주가 예측에 RNN을 사용한 이유는 무엇인가요? RNN은 **순서가 있는 데이터에서 특징을 추출하는 데 특화**되어 있어, 시간의 흐름에 따른 주가 변동 패턴을 학습하는 데 효과적이기 때문입니다.

1. RNN을 이용한 넷플릭스 주가 예측

RNN은 순서가 있는 시계열 데이터에서 특징을 추출하는 데 특화되어 있어 넷플릭스 주가 예측에 효과적으로 사용될 수 있다.



1.1. RNN의 개념 및 작동 방식

1. RNN(Recurrent Neural Network)이란?

- 가중치를 반복 사용하여 데이터로부터 순서 정보를 추출하는 신경망이다.
- 각 시점마다 추출된 특징들을 '은닉 상태'라고 부른다.
- 첫 번째 시점에는 이전 상태가 없으므로 랜덤하게 초기 은닉 상태를 정의해야 한다.
- 각 시점에서 동일한 가중치를 반복 사용하여 입력과 이전 은닉 상태를 바탕으로 현재 은닉 상태를 계산한다.

2. RNN이 시계열 데이터에 효과적인 이유

- 순서가 있는 데이터를 개별 시점(1일차, 2일차 등)으로 나누어 처리함으로써 각 시점의 정보를 활용한다.
- 만약 순서가 있는 데이터를 하나의 큰 데이터로 처리하면(예: MLP 사용), 데이터의 순서가 바뀌었을 때(ACB, OOD ACE) 모델이 이를 인식하지 못하는 단점이 발생한다.
- RNN은 순서 정보를 효과적으로 학습하여 이러한 문제를 해결한다.

1.2. 데이터 준비 및 전처리

1. 데이터 확인 및 이해

- 판다스 라이브러리를 사용하여 CSV 파일을 불러온다.
- CSV 파일은 쉼표로 구분된 데이터로, 날짜, 시가, 최고가, 최저가, 거래량, 종가 등의 정보를 포함한다.
- 본 예측에서는 날짜, 시가, 최고가, 최저가, 거래량(4가지 값)을 이용하여 내일의 종가를 예측한다.

2. PyTorch 데이터셋 구성

- `torch.utils.data.Dataset` 을 상속받아 데이터셋을 정의한다.
- CSV 파일에서 불러온 데이터를 입력 데이터(`data`)와 정답(`target`)으로 분리한다.
- 정규화:
 - 입력 데이터와 종가 데이터를 최대값으로 나누어 0과 1 사이의 값으로 스케일을 조정한다 (최대-최소 정규화).
 - 이는 데이터의 분포를 맞추기 위함이다.
- 시계열 데이터셋 구성:
 - 30일간의 데이터를 입력으로 사용하고, 31일째의 종가를 정답으로 사용한다.
 - 예를 들어, 1일부터 30일까지의 데이터를 사용하여 31일째의 종가를 예측한다.
 - 데이터의 마지막 날짜에는 다음 날의 종가가 존재하지 않으므로, 해당 부분까지의 데이터만 사용한다.
- `__len__` 메소드는 데이터셋의 총 개수를 반환한다.
- `__getitem__` 메소드는 특정 인덱스에 해당하는 입력 데이터와 정답 값을 반환한다.

1.3. RNN 모델 정의

1. 모델 구조 설계

- 여러 개의 RNN 층을 쌓아 순서에 대한 정보를 더 깊이 추출할 수 있다.
- 각 RNN 층은 서로 다른 가중치를 사용하여 다른 특징을 학습한다.
- 마지막에는 MLP 층을 사용하여 최종 예측값을 얻는다.
- 주가 예측은 회귀 문제이므로, MLP 층은 분류 층이 아닌 회귀 층으로 사용된다.

2. PyTorch 코드 구현

- `torch.nn.RNN` 클래스를 사용하여 RNN 층을 정의한다.
 - `input_size`: 입력 데이터의 차원 수
 - `hidden_size`: 은닉 상태의 차원 수
 - `num_layers`: 쌓을 RNN 층의 개수 (예: 5개)
 - `batch_first`: 배치가 먼저 오는 형태로 데이터를 처리할지 여부 (True로 설정 시 배치 크기가 첫 번째 차원이 됨)
- 주가 예측을 위한 MLP 층을 정의한다.
 - MLP 층은 1차원 벡터만 받을 수 있으므로, RNN의 출력을 평탄화(flatten)하거나 reshape하여 MLP에 입력해야 한다.
 - MLP 층의 입력 크기는 (배치 크기 * 은닉 상태 차원 수)가 된다.
 - 최종 출력은 예측하려는 종가 값이므로 1개의 뉴런을 가진다.
 - 활성화 함수로 ReLU를 사용한다.
- RNN 함수는 마지막 은닉 상태와 이전 은닉 상태를 동시에 반환한다.
- 초기 은닉 상태(`h_0`)는 RNN 계산을 위해 필요하며, 랜덤하게 초기화하거나 0으로 설정할 수 있다.
- `flatten` 은 데이터를 단순히 펼치는 반면, `reshape` 는 원하는 모양으로 변경할 수 있다.
- 모델 학습을 위해 모델, 옵티마이저, 손실 함수 등을 정의한다.

1.4. 모델 학습

1. 학습 과정 개요

- 데이터셋과 데이터 로더를 준비한다.

- 옵티마이저(예: Adam)와 손실 함수(예: MSELoss)를 설정한다.
- 학습은 200 에포크 동안 반복된다.

2. 학습 루프 상세

- 각 에포크마다 데이터 로더에서 배치 데이터를 가져온다.
- 초기 은닉 상태를 설정한다.
- 모델에 데이터를 입력하여 예측값을 계산한다.
 - RNN 입력 형태는 (층 개수, 배치 크기, 은닉 상태 차원 수) 순서로 맞춰야 한다.
 - 예측값은 실수형 텐서로 변환하고 디바이스로 보낸다.
- 예측값과 실제 정답 값으로 손실을 계산한다.
 - 정답 값도 실수형 텐서로 변환하고 디바이스로 보낸다.
- 계산된 손실을 바탕으로 역전파를 수행하고 옵티마이저를 통해 모델 가중치를 업데이트한다.
- 에포크별 손실을 기록하고 출력한다.

3. 학습 완료 및 저장

- 학습이 완료되면 모델을 저장한다.

1.5. 모델 성능 평가

1. 예측값 생성 및 비교 준비

- 학습된 모델을 사용하여 테스트 데이터셋에 대한 예측값을 생성한다.
- 예측값은 순서대로 리스트에 저장하여 그래프를 그릴 준비를 한다.
- 테스트 데이터셋의 정답 값도 불러온다.
- 초기 은닉 상태를 설정한다.

2. 성능 지표 확인

- 전체 손실을 데이터 로더의 개수로 나누어 평균 손실을 계산한다.
- 계산된 평균 손실은 약 0.00125이다.
- 이 오차 값은 0과 1 사이의 정규화된 주가 범위를 고려할 때 아주 작다고 보기는 어렵다.

3. 실제 주가와 예측 주가 시각화

- `matplotlib.pyplot`을 사용하여 실제 주가와 모델이 예측한 주가를 그래프로 그린다.

- 주황색 선은 실제 주가를 나타내고, 파란색 선은 모델이 예측한 주가를 나타낸다.
- 그래프를 통해 모델이 실제 주가의 변동성을 완전히 따라가지는 못하지만, 대략적인 추세는 잘 따라가고 있음을 확인할 수 있다.
- 이는 모델이 엉터리로 학습된 것이 아니며, 꽤 잘 이루어졌음을 시사한다.